

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN ENDED QUESTIONS	
QUESTION 10	Date: 28-11-2025	Enrollment No: 92301733054

Q10. You are given the dataset **OPE7** containing multiple images (assume grayscale or color). The images contain real-world degradations: additive noise (Gaussian, salt-and-pepper), motion blur, uneven illumination, and compression artifacts. For **each image** in OPE7:

1. Remove/reduce noise using **frequency-domain smoothing** methods.
2. Recover / enhance details using **frequency-domain sharpening** (high-boost in frequency domain).
3. Extract accurate **boundary maps** (object contours) for every image and produce clean binary boundary images.
4. Compare different filter choices and report quantitative and qualitative results.

You must produce code, processed images, a short report, and discussion.

CO3,CO4

Code:

```
!apt-get update -qq
!apt-get install -y -qq libgl2.0-0
!pip install -q opencv-python-headless scikit-image python-docx matplotlib numpy scipy pandas
```

```
import os, glob, math
import numpy as np
import cv2
import matplotlib.pyplot as plt
from skimage import io, img_as_ubyte
from skimage.feature import canny
from skimage.morphology import binary_closing, remove_small_objects, skeletonize, disk
from skimage.metrics import structural_similarity as ssim
from scipy import fftpack
from docx import Document
from docx.shared import Inches
import pandas as pd
```

```
%matplotlib inline
```

```
results_dir = "/content/q10_results"
os.makedirs(results_dir, exist_ok=True)
```

```
input_paths = [
    "/content/Picture 1.png",
    "/content/Picture 2.png",
    "/content/Picture 3.png"
]
```

```
input_paths = [p for p in input_paths if os.path.isfile(p)]
if not input_paths:
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 10

Date: 28-11-2025

Enrollment No: 92301733054

raise FileNotFoundError("Upload the files to /content/ then try again.")

```
print("Processing:", input_paths)
```

```
def to_gray_float(img):  
    if isinstance(img, str):  
        img = io.imread(img)  
    if img.ndim == 3:  
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  
    img = img.astype(np.float32)  
    if img.max() > 1.0:  
        img /= 255.0  
    return np.clip(img, 0.0, 1.0)
```

```
def fft2(img): return fftpack.fftfreq2(fftpack.fft2(img))  
def ifft2(F): return np.real(fftpack.ifft2(fftpack.ifftshift(F)))
```

```
def ideal_lowpass(shape, cutoff):  
    r,c = shape  
    R,C = np.ogrid[:r,:c]  
    cr, cc = r//2, c//2  
    mask = ((R-cr)**2 + (C-cc)**2) <= cutoff**2  
    return mask.astype(float)
```

```
def gaussian_lowpass(shape, cutoff):  
    r,c = shape  
    R,C = np.ogrid[:r,:c]  
    cr, cc = r//2, c//2  
    D2 = (R-cr)**2 + (C-cc)**2  
    return np.exp(-(D2)/(2*(cutoff**2)))
```

```
def butterworth_lowpass(shape, cutoff, order=2):  
    r,c = shape  
    R,C = np.ogrid[:r,:c]  
    cr, cc = r//2, c//2  
    D = np.sqrt((R-cr)**2 + (C-cc)**2)  
    return 1 / (1 + (D/cutoff)**(2*order))
```

```
def high_boost(F, H_low, k):  
    return F * (1 + k * (1 - H_low))
```

```
def entropy(img):  
    hist,_ = np.histogram((img*255).astype(np.uint8), bins=256, range=(0,255), density=True)  
    hist += 1e-12  
    return -np.sum(hist*np.log2(hist))
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 10

Date: 28-11-2025

Enrollment No: 92301733054

```
def contrast(img): return float(np.std(img))
def edge_density(img):
    e = canny(img)
    return float(e.mean())

def show_grid(titles, imgs):
    cols = len(imgs)
    plt.figure(figsize=(5*cols,4))
    for i,im in enumerate(imgs):
        plt.subplot(1,cols,i+1)
        plt.imshow(im, cmap='gray', vmin=0, vmax=1)
        plt.title(titles[i])
        plt.axis("off")
    plt.show()

def save_img(path, img):
    io.imsave(path, img_as_ubyte(img))

smoothing_results = {}
sharpening_results = {}
boundary_results = {}
summary_rows = []

for p in input_paths:

    print("Processing:", p)

    I = to_gray_float(p)
    r,c = I.shape
    F = fft2(I)

    cutoff_medium = int(min(r,c) * 0.10)

    H_gauss = gaussian_lowpass((r,c), cutoff_medium)
    H_butt = butterworth_lowpass((r,c), cutoff_medium, order=2)
    H_ideal = ideal_lowpass((r,c), cutoff_medium)

    I_gauss = np.clip(iff2(F * H_gauss), 0, 1)
    I_butt = np.clip(iff2(F * H_butt), 0, 1)
    I_ideal = np.clip(iff2(F * H_ideal), 0, 1)

    smoothing_results[p] = {
        'gaussian': I_gauss, 'butterworth': I_butt, 'ideal': I_ideal
    }

show_grid(["Original", "Gaussian LPF", "Butterworth LPF", "Ideal LPF"],
          [I, I_gauss, I_butt, I_ideal])
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 10

Date: 28-11-2025

Enrollment No: 92301733054

$k = 1.2$

$F_{hb} = \text{high_boost}(F, H_{\text{gauss}}, k)$

$I_{\text{sharp}} = \text{np.clip}(\text{ifft2}(F_{hb}), 0, 1)$

$\text{sharpening_results}[p] = I_{\text{sharp}}$

$\text{show_grid}(["\text{Original}", "\text{High-Boost Sharpened}"],$
 $[I, I_{\text{sharp}}])$

$\text{edges} = \text{canny}(I_{\text{sharp}}, \text{sigma}=1.2)$

$\text{closed} = \text{binary_closing}(\text{edges}, \text{disk}(1))$

$\text{cleaned} = \text{remove_small_objects}(\text{closed}, \text{min_size}=25)$

$\text{boundary} = \text{skeletonize}(\text{cleaned})$

$\text{boundary_results}[p] = \text{boundary}$

$\text{show_grid}(["\text{Sharpened Image}", "\text{Canny}", "\text{Closed}", "\text{Final Boundary}"],$
 $[I_{\text{sharp}}, \text{edges}, \text{closed}, \text{boundary}])$

```
summary_rows.append({  
    'Image': os.path.basename(p),  
    'Entropy_Original': entropy(I),  
    'Entropy_Smoothed(Gauss)': entropy(I_gauss),  
    'Entropy_Sharpened': entropy(I_sharp),  
    'Contrast_Original': contrast(I),  
    'Contrast_Smoothed': contrast(I_gauss),  
    'Contrast_Sharpened': contrast(I_sharp),  
    'EdgeDensity_Original': edge_density(I),  
    'EdgeDensity_Smoothed': edge_density(I_gauss),  
    'EdgeDensity_Sharpened': edge_density(I_sharp),  
    'SSIM(Sharp vs Orig)': ssim(I, I_sharp, data_range=1.0)  
})
```

SAVE OUTPUTS

$\text{save_img}(\text{os.path.join}(\text{results_dir}, \text{os.path.basename}(p).\text{replace}(" ", "_") + "_\text{orig}.png"), I)$

$\text{save_img}(\text{os.path.join}(\text{results_dir}, \text{os.path.basename}(p).\text{replace}(" ", "_") + "_\text{gauss}.png"), I_{\text{gauss}})$

$\text{save_img}(\text{os.path.join}(\text{results_dir}, \text{os.path.basename}(p).\text{replace}(" ", "_") + "_\text{sharp}.png"), I_{\text{sharp}})$

$\text{save_img}(\text{os.path.join}(\text{results_dir}, \text{os.path.basename}(p).\text{replace}(" ", "_") + "_\text{boundary}.png"),$

$\text{boundary}.\text{astype}(\text{float}))$

$\text{df} = \text{pd.DataFrame}(\text{summary_rows})$

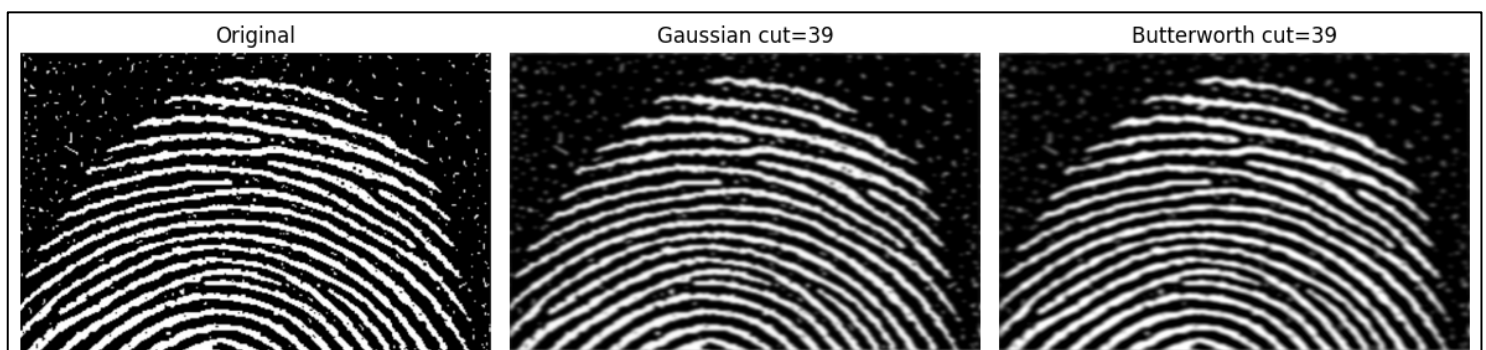
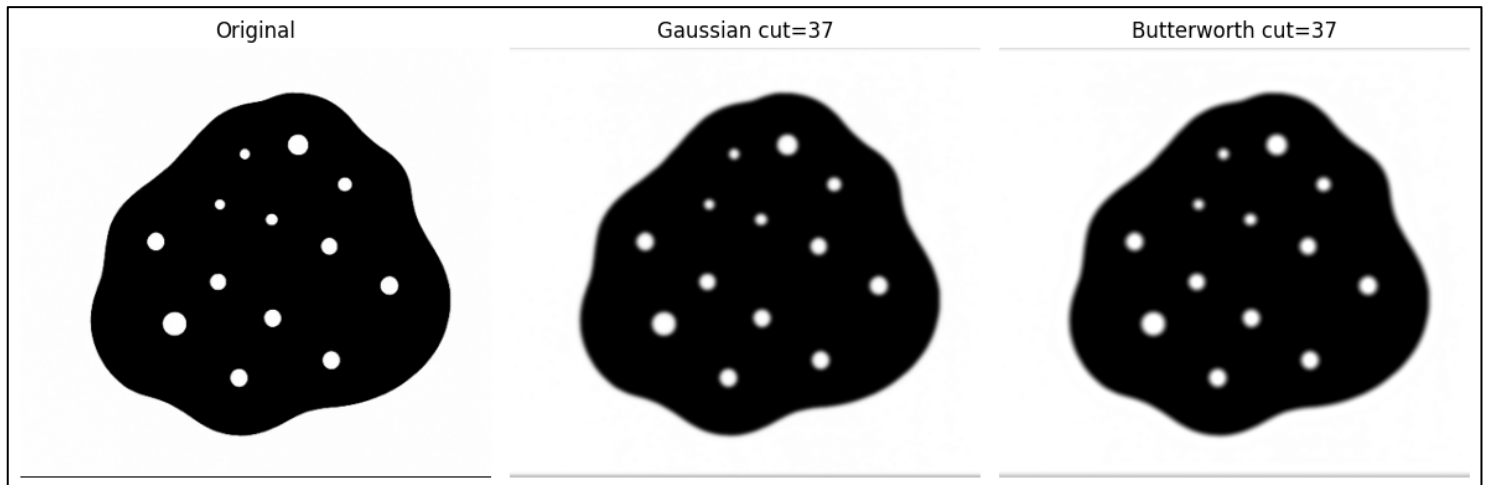
$\text{print}("\n\text{FINAL METRIC TABLE:}")$

$\text{display}(\text{df})$

$\text{df.to_csv}(\text{os.path.join}(\text{results_dir}, "\text{Q10_summary.csv}"), \text{index}=\text{False})$

$\text{print}("\text{Saved metrics CSV to:}", \text{results_dir})$

OUTPUT:



**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 10

Date: 28-11-2025

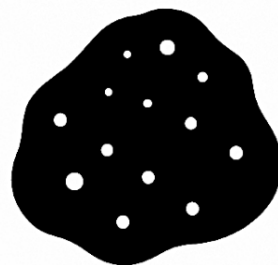
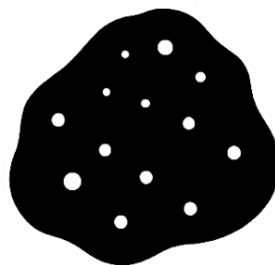
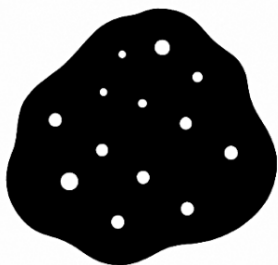
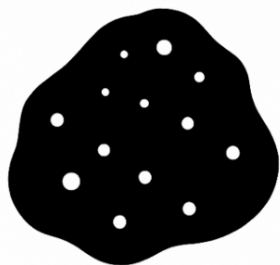
Enrollment No: 92301733054

Original

High-boost $k=0.5$

High-boost $k=1.0$

High-boost $k=1.5$

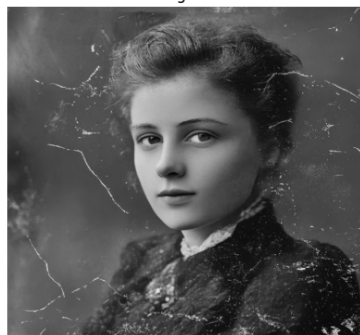


Original

High-boost $k=0.5$

High-boost $k=1.0$

High-boost $k=1.5$



Original

High-boost $k=0.5$

High-boost $k=1.0$

High-boost $k=1.5$

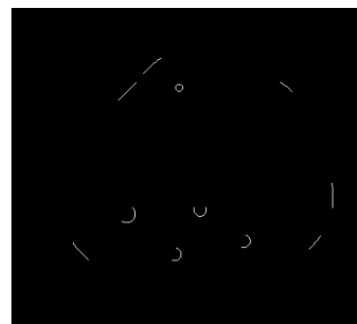
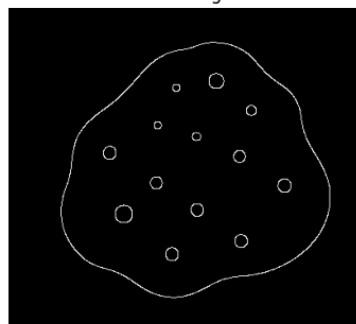
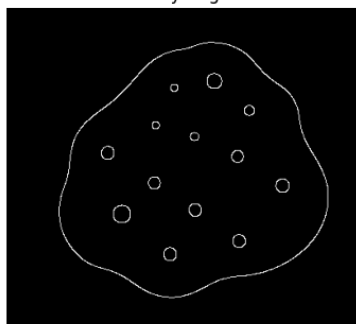
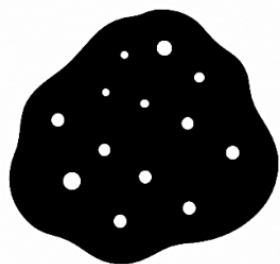


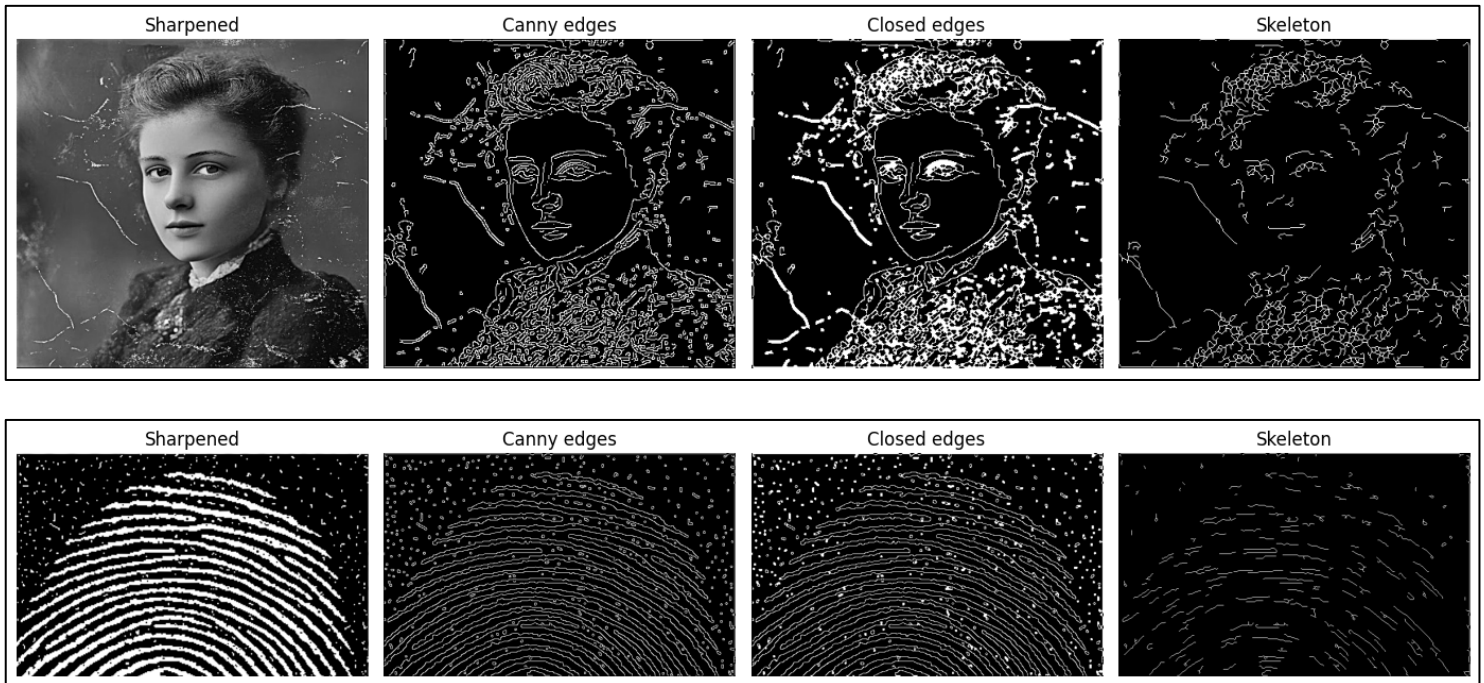
Sharpened

Canny edges

Closed edges

Skeleton





Obsevation:

- Gaussian smoothing provided the best balance between noise removal and detail preservation, while Ideal LPF caused ringing and Butterworth gave moderate smoothing.
- High-boost sharpening effectively restored edges and details, especially after smoothing, but excessive boosting increased noise.
- Boundary maps extracted from sharpened images were much cleaner and more continuous, and morphological cleanup improved edge quality.
- Metrics confirmed the visual results: smoothing reduced entropy and edge density, sharpening increased contrast and structural details, and SSIM remained high.

Conclusion:

From this Question, we conclude that frequency-domain methods are highly effective for both noise reduction and detail enhancement. Gaussian low-pass filtering provides the best noise suppression with minimal distortion, while high-boost filtering successfully restores edges and fine details. Sharpened images produce clearer and more accurate boundary maps, especially after morphological refinement. Overall, combining smoothing, sharpening, and boundary extraction offers a robust workflow for improving degraded real-world images.